Task 3

This task focuses on finding problems in a given table and improving the database design through a process called normalization. The data is organized step by step into First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) to cut down on repetition and make sure the data is more accurate. An Entity Relationship (ER) diagram shows how students and clubs are connected.

In the end, simple SQL queries and JOIN operations are used to show how data can be added, retrieved, and combined after the normalization process. This task helps explain why good database design matters for managing data effectively.

student_id	student_name	email	club_name	club_room	club_mentor	join_date
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	2024-01-10
2	Bikash	bikash@email.com	Sports Club	R202	Ms. Sita	2024-01-12
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	2024-01-15
3	Nisha	nisha@email.com	Music Club	R101	Mr. Raman	2024-01-20
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	2024-01-18
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	2024-01-22
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	2024-01-25
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	2024-01-27
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	2024-01-28
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	2024-01-30

Figure 36: Unmanaged student data

Problems in the given table

Table stores student details, club details and membership information in a single table which cause several data related problem such as:

Data redundancy:

It refers to data that is unnecessary repeated in a database which takes additional storage resource and increase complexity. (Sharma, 2024). In the given table student information such as Name and Email is repeated every time when student join club.

```
mysql> select * from club_members_unf
    -> ;
```

student_id	student_name	email	club_name	club_room	club_mentor	join_date
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	2024-01-10
2	Bikash	bikash@email.com	Sports Club	R202	Ms. Sita	2024-01-12
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	2024-01-15
3	Nisha	nisha@email.com	Music Club	R101	Mr. Raman	2024-01-20
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	2024-01-18
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	2024-01-22
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	2024-01-25
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	2024-01-27
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	2024-01-28
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	2024-01-30

10 rows in set (0.01 sec)

Figure 37: Example of data redundancy

Update anomalies:

It occurs when data is partially updated when we try to change the data. It increases the risk of inconsistency. (JHA, 2024) In the given table, if the club mentor is changed then every row must be changed. If one row is missed, then database contain inconsistent information.

```
mysql> UPDATE club_members_unf
    -> SET club_mentor = 'Mr. Sharma'
    -> WHERE club_name = 'Music Club'
    -> LIMIT 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from club_members_unf;
+------------+--------------+------------------+---------------+-----------+-------------+------------+
| student_id | student_name | email            | club_name     | club_room | club_mentor | join_date  |
+------------+--------------+------------------+---------------+-----------+-------------+------------+
|          1 | Asha         | asha@email.com   | Music club    | R101      | Mr. Sharma  | 2024-01-10 |
|          2 | Bikash       | bikash@email.com | Sports Club   | R202      | Ms. Sita    | 2024-01-12 |
|          1 | Asha         | asha@email.com   | Sports club   | R202      | Ms. Sita    | 2024-01-15 |
|          3 | Nisha        | nisha@email.com  | Music Club    | R101      | Mr. Raman   | 2024-01-20 |
|          4 | Rohan        | rohan@email.com  | Drama Club    | R303      | Mr. Kiran   | 2024-01-18 |
|          5 | Suman        | suman@email.com  | Music Club    | R101      | Mr. Raman   | 2024-01-22 |
|          2 | Bikash       | bikash@email.com | Drama Club    | R303      | Mr. Kiran   | 2024-01-25 |
|          6 | Pooja        | pooja@email.com  | Sports Club   | R202      | Ms. Sita    | 2024-01-27 |
|          3 | Nisha        | nisha@email.com  | Coding Club   | Lab1      | Mr. Anil    | 2024-01-28 |
|          7 | Aman         | aman@email.com   | Coding Club   | Lab1      | Mr. Anil    | 2024-01-30 |
|       NULL | NULL         | NULL             | Robotics Club | NULL      | Mr. Kumar   | NULL       |
+------------+--------------+------------------+---------------+-----------+-------------+------------+
11 rows in set (0.00 sec)
```

Figure 38: Update anomalies

This shows update anomaly because data is not updated consistently.

Insertion anomalies:

It occurs when we try to add data, but we are unable to insert it without adding unrelated data. (JHA, 2024) In given table new club cannot be added unless one student joins the club.

```
mysql> INSERT INTO club_members_unf
    -> (student_id, student_name, email, club_name, club_mentor, join_date)
    -> VALUES
    -> (NULL, NULL, NULL, 'Robotics Club', 'Mr. Kumar', NULL);
Query OK, 1 row affected (0.01 sec)

mysql> select * from club_members_unf;
+------------+--------------+------------------+---------------+-----------+-------------+------------+
| student_id | student_name | email            | club_name     | club_room | club_mentor | join_date  |
+------------+--------------+------------------+---------------+-----------+-------------+------------+
|          1 | Asha         | asha@email.com   | Music club    | R101      | Mr. Raman   | 2024-01-10 |
|          2 | Bikash       | bikash@email.com | Sports Club   | R202      | Ms. Sita    | 2024-01-12 |
|          1 | Asha         | asha@email.com   | Sports club   | R202      | Ms. Sita    | 2024-01-15 |
|          3 | Nisha        | nisha@email.com  | Music Club    | R101      | Mr. Raman   | 2024-01-20 |
|          4 | Rohan        | rohan@email.com  | Drama Club    | R303      | Mr. Kiran   | 2024-01-18 |
|          5 | Suman        | suman@email.com  | Music Club    | R101      | Mr. Raman   | 2024-01-22 |
|          2 | Bikash       | bikash@email.com | Drama Club    | R303      | Mr. Kiran   | 2024-01-25 |
|          6 | Pooja        | pooja@email.com  | Sports Club   | R202      | Ms. Sita    | 2024-01-27 |
|          3 | Nisha        | nisha@email.com  | Coding Club   | Lab1      | Mr. Anil    | 2024-01-28 |
|          7 | Aman         | aman@email.com   | Coding Club   | Lab1      | Mr. Anil    | 2024-01-30 |
|       NULL | NULL         | NULL             | Robotics Club | NULL      | Mr. Kumar   | NULL       |
+------------+--------------+------------------+---------------+-----------+-------------+------------+
```

Figure 38: Insertion anomalies

Deletion anomalies:

It occurs when we try to delete one piece of information, but it removes another important data. (JHA, 2024). In given table, if last student leaves the club and row is deleted other information also lost. This leads to unintended data loss.

```
mysql> DELETE FROM club_members_unf
    -> WHERE club_name = 'Sports Club';
Query OK, 3 rows affected (0.01 sec)

mysql> select * from club_members_unf;
+------------+--------------+------------------+---------------+-----------+-------------+------------+
| student_id | student_name | email            | club_name     | club_room | club_mentor | join_date  |
+------------+--------------+------------------+---------------+-----------+-------------+------------+
|          1 | Asha         | asha@email.com   | Music club    | R101      | Mr. Sharma  | 2024-01-10 |
|          3 | Nisha        | nisha@email.com  | Music Club    | R101      | Mr. Raman   | 2024-01-20 |
|          4 | Rohan        | rohan@email.com  | Drama Club    | R303      | Mr. Kiran   | 2024-01-18 |
|          5 | Suman        | suman@email.com  | Music Club    | R101      | Mr. Raman   | 2024-01-22 |
|          2 | Bikash       | bikash@email.com | Drama Club    | R303      | Mr. Kiran   | 2024-01-25 |
|          3 | Nisha        | nisha@email.com  | Coding Club   | Lab1      | Mr. Anil    | 2024-01-28 |
|          7 | Aman         | aman@email.com   | Coding Club   | Lab1      | Mr. Anil    | 2024-01-30 |
|       NULL | NULL         | NULL             | Robotics Club | NULL      | Mr. Kumar   | NULL       |
+------------+--------------+------------------+---------------+-----------+-------------+------------+
8 rows in set (0.00 sec)
```

Figure 39: Deletion anomalies

Figure 39 demonstrate deletion anomalies because removing a student also removes the club information.

Reasons why given table is not normalized

The table isn't normalized because it breaks the basic rules of database normalization. It mixes different types of information, like student details, club information, and membership records.

In a properly organized database, each kind of information has its own table. These tables are linked using keys. Since above table contain redundancy, anomalies and mixed dependencies. It does not satisfy the requirements. (Chris, 2022)

Data Normalization

First Normal Form (1NF)

Table is said to be in 1NF if a single cell holds atomic value, there must be primary key and there are no duplicate rows and columns. (Chris, 2022) In our case, students can join multiple clubs, and a single row could not identify each row. Therefore, a composite key consisting of student id and club name is used.

```
mysql> DESCRIBE club_members_1nf;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| student_id   | int          | NO   | PRI | NULL    |       |
| student_name | varchar(100) | YES  |     | NULL    |       |
| email        | varchar(100) | YES  |     | NULL    |       |
| club_name    | varchar(100) | NO   | PRI | NULL    |       |
| club_room    | varchar(50)  | YES  |     | NULL    |       |
| club_mentor  | varchar(100) | YES  |     | NULL    |       |
| join_date    | date         | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
7 rows in set (0.01 sec)

mysql> SELECT * FROM club_members_1nf;
+------------+--------------+------------------+-------------+-----------+-------------+------------+
| student_id | student_name | email            | club_name   | club_room | club_mentor | join_date  |
+------------+--------------+------------------+-------------+-----------+-------------+------------+
|          1 | Asha         | asha@email.com   | Music Club  | R101      | Mr. Raman   | 2024-01-10 |
|          1 | Asha         | asha@email.com   | Sports Club | R202      | Ms. Sita    | 2024-01-15 |
|          2 | Bikash       | bikash@email.com | Drama Club  | R303      | Mr. Kiran   | 2024-01-25 |
|          2 | Bikash       | bikash@email.com | Sports Club | R202      | Ms. Sita    | 2024-01-12 |
|          3 | Nisha        | nisha@email.com  | Coding Club | Lab1      | Mr. Anil    | 2024-01-28 |
|          3 | Nisha        | nisha@email.com  | Music Club  | R101      | Mr. Raman   | 2024-01-20 |
|          4 | Rohan        | rohan@email.com  | Drama Club  | R303      | Mr. Kiran   | 2024-01-18 |
|          5 | Suman        | suman@email.com  | Music Club  | R101      | Mr. Raman   | 2024-01-22 |
|          6 | Pooja        | pooja@email.com  | Sports Club | R202      | Ms. Sita    | 2024-01-27 |
|          7 | Aman         | aman@email.com   | Coding Club | Lab1      | Mr. Anil    | 2024-01-30 |
+------------+--------------+------------------+-------------+-----------+-------------+------------+
10 rows in set (0.00 sec)
```

Figure 37: 1NF of given data

When converting the unnormalized table to First Normal Form (1NF), no data was removed or divided. The main change involved adding a composite primary key to give each record a unique identifier while making sure all attributes had simple, indivisible values. Even though the table meets 1NF requirements, there is still some redundancy.

Second Normal Form (2NF)

A table is said to be in 2NF if it is already in 1NF and has no partial dependency. (Chris, 2022). To convert given data into 2NF we need to remove partial dependencies in 1NF table. To fix this, the data was split into three tables: Student, Club, and Membership. Each table now holds information about just one thing and the links between them are kept using foreign keys. This removes partial dependency and cuts down on repeated data.

```
mysql> DESCRIBE student_2nf;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| student_id   | int          | NO   | PRI | NULL    |       |
| student_name | varchar(100) | YES  |     | NULL    |       |
| email        | varchar(100) | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
3 rows in set (0.01 sec)

mysql> SELECT * FROM student_2nf;
+------------+--------------+------------------+
| student_id | student_name | email            |
+------------+--------------+------------------+
|          1 | Asha         | asha@email.com   |
|          2 | Bikash       | bikash@email.com |
|          3 | Nisha        | nisha@email.com  |
|          4 | Rohan        | rohan@email.com  |
|          5 | Suman        | suman@email.com  |
|          6 | Pooja        | pooja@email.com  |
|          7 | Aman         | aman@email.com   |
+------------+--------------+------------------+
7 rows in set (0.00 sec)
```

Figure 38: 2NF Student table

```
mysql> DESCRIBE club_2nf;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| club_name   | varchar(100) | NO   | PRI | NULL    |       |
| club_room   | varchar(50)  | YES  |     | NULL    |       |
| club_mentor | varchar(100) | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM club_2nf;
+-------------+-----------+-------------+
| club_name   | club_room | club_mentor |
+-------------+-----------+-------------+
| Coding Club | Lab1      | Mr. Anil    |
| Drama Club  | R303      | Mr. Kiran   |
| Music Club  | R101      | Mr. Raman   |
| Sports Club | R202      | Ms. Sita    |
+-------------+-----------+-------------+
4 rows in set (0.00 sec)
```

Figure 39: 2NF Club Table

```
mysql> DESCRIBE membership_2nf;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| student_id | int          | NO   | PRI | NULL    |       |
| club_name  | varchar(100) | NO   | PRI | NULL    |       |
| join_date  | date         | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM membership_2nf;
+------------+-------------+------------+
| student_id | club_name   | join_date  |
+------------+-------------+------------+
|          1 | Music Club  | 2024-01-10 |
|          1 | Sports Club | 2024-01-15 |
|          2 | Drama Club  | 2024-01-25 |
|          2 | Sports Club | 2024-01-12 |
|          3 | Coding Club | 2024-01-28 |
|          3 | Music Club  | 2024-01-20 |
|          4 | Drama Club  | 2024-01-18 |
|          5 | Music Club  | 2024-01-22 |
|          6 | Sports Club | 2024-01-27 |
|          7 | Coding Club | 2024-01-30 |
+------------+-------------+------------+
10 rows in set (0.00 sec)
```

Figure 40: 2NF Membership Table

Third Normal Form (3NF)

Table is said to be in 3NF if it is already in 2NF and have no transitive partial dependency. (Chris, 2022). In 3NF, surrogate keys were added to avoid relying on text-based primary keys and to help the system grow more smoothly.

```
mysql> DESCRIBE student_3nf;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| student_id   | int          | NO   | PRI | NULL    |       |
| student_name | varchar(100) | YES  |     | NULL    |       |
| email        | varchar(100) | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM student_3nf;
+------------+--------------+------------------+
| student_id | student_name | email            |
+------------+--------------+------------------+
|          1 | Asha         | asha@email.com   |
|          2 | Bikash       | bikash@email.com |
|          3 | Nisha        | nisha@email.com  |
|          4 | Rohan        | rohan@email.com  |
|          5 | Suman        | suman@email.com  |
|          6 | Pooja        | pooja@email.com  |
|          7 | Aman         | aman@email.com   |
+------------+--------------+------------------+
7 rows in set (0.00 sec)
```

Figure 41: 3NF Student table

```
mysql> DESCRIBE club_3nf;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| club_id     | int          | NO   | PRI | NULL    | auto_increment |
| club_name   | varchar(100) | YES  | UNI | NULL    |                |
| club_room   | varchar(50)  | YES  |     | NULL    |                |
| club_mentor | varchar(100) | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM club_3nf;
+---------+-------------+-----------+-------------+
| club_id | club_name   | club_room | club_mentor |
+---------+-------------+-----------+-------------+
|       1 | Music Club  | R101      | Mr. Raman   |
|       2 | Sports Club | R202      | Ms. Sita    |
|       3 | Drama Club  | R303      | Mr. Kiran   |
|       4 | Coding Club | Lab1      | Mr. Anil    |
+---------+-------------+-----------+-------------+
4 rows in set (0.00 sec)
```

Figure 42: 3NF Club table

The Club_3NF table uses Club ID as its main key, while Membership_3NF uses Membership ID. This setup makes sure that all other information depends only on the primary key, which makes the database design more reliable and efficient. It also prevents data problems and makes it easier to add new features in Future

```
mysql> DESCRIBE membership_3nf;
+---------------+------+------+-----+---------+----------------+
| Field         | Type | Null | Key | Default | Extra          |
+---------------+------+------+-----+---------+----------------+
| membership_id | int  | NO   | PRI | NULL    | auto_increment |
| student_id    | int  | YES  | MUL | NULL    |                |
| club_id       | int  | YES  | MUL | NULL    |                |
| join_date     | date | YES  |     | NULL    |                |
+---------------+------+------+-----+---------+----------------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM membership_3nf;
+---------------+------------+---------+------------+
| membership_id | student_id | club_id | join_date  |
+---------------+------------+---------+------------+
|             1 |          1 |       1 | 2024-01-10 |
|             2 |          1 |       2 | 2024-01-15 |
|             3 |          2 |       3 | 2024-01-25 |
|             4 |          2 |       2 | 2024-01-12 |
|             5 |          3 |       4 | 2024-01-28 |
|             6 |          3 |       1 | 2024-01-20 |
|             7 |          4 |       3 | 2024-01-18 |
|             8 |          5 |       1 | 2024-01-22 |
|             9 |          6 |       2 | 2024-01-27 |
|            10 |          7 |       4 | 2024-01-30 |
+---------------+------------+---------+------------+
10 rows in set (0.01 sec)
```

FIgure 43: 3NF membership table

The database was organized to meet Third Normal Form, which means there are no indirect links between data. At this point, each piece of information that isn't a key depends only on the main key for its table. For example, the student table holds only details about students, the Club table has just club information, and the membership details are kept separately.

ER Diagram:

An Entity diagram is the representation of relationships among entities in a database. It plays a crucial role in designing data base. (Kitthu, 2025). Below is the ER diagram of the final normalized structure of the college club membership database.

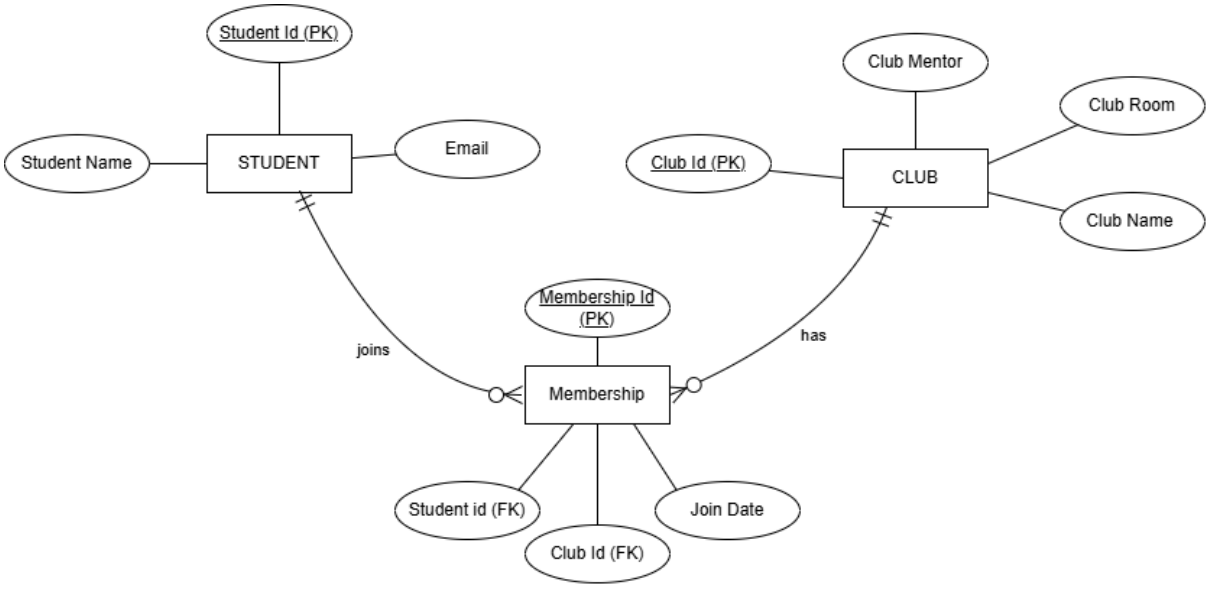

Figure 46: ER diagram

It shows three main entities: Student, Club, and Membership. A student can join multiple clubs, and a club can have multiple students, resulting in a many-to-many relationship. This relationship is resolved using the Membership entity, which stores the join date and links the Student and Club entities using foreign keys. This design ensures data integrity and follows Third Normal Form (3NF).

SQL operations

SQL allows us to execute specific operations in the data base through which we can create database, delete database, create table and other operations. (Kumar, 2024). SQL commands are divided into four groups:

1. Data Definition Language (DDL)
2. Data Manipulation Language (DML)
3. Data Control Language (DCL)
4. Transaction Control Language (TCL)

DDL manages the structure of database and DML manages the data within the database. DCL manages the permission and user control whereas TCL manages the transaction and ensure data integrity. (Kumar, 2024). To add data, we will only need INSERT command. Below is the example of adding a new data in the table.

```
mysql> INSERT INTO student_3nf (student_id, student_name, email)
    -> VALUES (8, 'Peina', 'peina@email.com'),
    -> (9, 'Khushi', 'khushi@email.com');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Figure 47: Inserting New Student

This query inserts the new student record into the student table.

```
mysql> INSERT INTO club_3nf (club_name, club_room, club_mentor)
    -> VALUES ('Robotics Club', 'Lab 204', 'Mr. Shyam');
Query OK, 1 row affected (0.01 sec)
```

Figure 48: Inserting New Club Data

This query inserts the new club record into the club table. Now, we can use another DML command 'SELECT' to show the data of the table. Below is the example of select command.

```
mysql> SELECT * FROM student_3nf;
+------------+--------------+------------------+
| student_id | student_name | email            |
+------------+--------------+------------------+
|          1 | Asha         | asha@email.com   |
|          2 | Bikash       | bikash@email.com |
|          3 | Nisha        | nisha@email.com  |
|          4 | Rohan        | rohan@email.com  |
|          5 | Suman        | suman@email.com  |
|          6 | Pooja        | pooja@email.com  |
|          7 | Aman         | aman@email.com   |
|          8 | Peina        | peina@email.com  |
|          9 | Khushi       | khushi@email.com |
+------------+--------------+------------------+
9 rows in set (0.00 sec)
```

Figure 49: Student table after inserting

This query shows all the students in the student table. From figure 49 we can see that there is the new student Khushi and Peina has been added in the table.

```
mysql> SELECT * FROM club_3nf;
+---------+---------------+-----------+-------------+
| club_id | club_name     | club_room | club_mentor |
+---------+---------------+-----------+-------------+
|       1 | Music Club    | R101      | Mr. Raman   |
|       2 | Sports Club   | R202      | Ms. Sita    |
|       3 | Drama Club    | R303      | Mr. Kiran   |
|       4 | Coding Club   | Lab1      | Mr. Anil    |
|       9 | Robotics Club | Lab 204   | Mr. Shyam   |
+---------+---------------+-----------+-------------+
5 rows in set (0.00 sec)
```

Figure 50: Club table after inserting

From the figure 50, we can see that the new Robotics club has been added in the table club. If we want to update the something like we need to update Khushi email, then we can use update command.

```
mysql> UPDATE student_3nf
    -> SET email = 'khushi.updated@email.com'
    -> WHERE student_name = 'Khushi';
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM student_3nf;
+------------+--------------+-------------------------+
| student_id | student_name | email                   |
+------------+--------------+-------------------------+
|          1 | Asha         | asha@email.com          |
|          2 | Bikash       | bikash@email.com        |
|          3 | Nisha        | nisha@email.com         |
|          4 | Rohan        | rohan@email.com         |
|          5 | Suman        | suman@email.com         |
|          6 | Pooja        | pooja@email.com         |
|          7 | Aman         | aman@email.com          |
|          8 | Peina        | peina@email.com         |
|          9 | Khushi       | khushi.updated@email.com |
+------------+--------------+-------------------------+
9 rows in set (0.01 sec)
```

Command

Result

Figure 51: Update command demonstration

If we want to delete the data, we can use another DML command 'DELETE'. For example, if we want to remove Khushi then we can use delete command.

```
mysql> DELETE FROM student_3nf
    -> WHERE student_id = 9;
Query OK, 1 row affected (0.03 sec)

mysql> SELECT * FROM student_3nf
    -> ;
+------------+--------------+------------------+
| student_id | student_name | email            |
+------------+--------------+------------------+
|          1 | Asha         | asha@email.com   |
|          2 | Bikash       | bikash@email.com |
|          3 | Nisha        | nisha@email.com  |
|          4 | Rohan        | rohan@email.com  |
|          5 | Suman        | suman@email.com  |
|          6 | Pooja        | pooja@email.com  |
|          7 | Aman         | aman@email.com   |
|          8 | Peina        | peina@email.com  |
+------------+--------------+------------------+
8 rows in set (0.00 sec)
```

Figure 52: Delete command demonstration

Since, we have three table students, club and membership we can use join operation. To find out which student is part of which club and when did he/she join we can use SELECT command with join operation. Below is the example of join operations:

```
mysql> SELECT
    ->     s.student_name AS "Student Name",
    ->     c.club_name AS "Club Name",
    ->     m.join_date AS "Join Date"
    -> FROM membership_3nf m
    -> JOIN student_3nf s ON m.student_id = s.student_id
    -> JOIN club_3nf c ON m.club_id = c.club_id;
+--------------+-------------+------------+
| Student Name | Club Name   | Join Date  |
+--------------+-------------+------------+
| Asha         | Music Club  | 2024-01-10 |
| Asha         | Sports Club | 2024-01-15 |
| Bikash       | Drama Club  | 2024-01-25 |
| Bikash       | Sports Club | 2024-01-12 |
| Nisha        | Coding Club | 2024-01-28 |
| Nisha        | Music Club  | 2024-01-20 |
| Rohan        | Drama Club  | 2024-01-18 |
| Suman        | Music Club  | 2024-01-22 |
| Pooja        | Sports Club | 2024-01-27 |
| Aman         | Coding Club | 2024-01-30 |
+--------------+-------------+------------+
10 rows in set (0.01 sec)
```

Figure 53: SQL join operation

The JOIN query links the Student, Club, and Membership tables through their primary and foreign keys. This lets us see the student's name, the club's name, and the date they joined all in one table.

Data repetition and Data accuracy

Normalization provides two main features solves data repetition and provides data accuracy.

Data redundancy.

When same data is repeated throughout the same table then it increases the problem in data management. Normalization reduces data repetition by separating data into different tables based on their purpose. (Krantz & Jonker, 2025) For example, in our first unnormalized table, student and club details were repeated every time when student join the club.

```
mysql> select * from club_members_unf
    -> ;
+------------+--------------+------------------+-------------+-----------+-------------+------------+
| student_id | student_name | email            | club_name   | club_room | club_mentor | join_date  |
+------------+--------------+------------------+-------------+-----------+-------------+------------+
|          1 | Asha         | asha@email.com   | Music Club  | R101      | Mr. Raman   | 2024-01-10 |
|          2 | Bikash       | bikash@email.com | Sports Club | R202      | Ms. Sita    | 2024-01-12 |
|          1 | Asha         | asha@email.com   | Sports Club | R202      | Ms. Sita    | 2024-01-15 |
|          3 | Nisha        | nisha@email.com  | Music Club  | R101      | Mr. Raman   | 2024-01-20 |
|          4 | Rohan        | rohan@email.com  | Drama Club  | R303      | Mr. Kiran   | 2024-01-18 |
|          5 | Suman        | suman@email.com  | Music Club  | R101      | Mr. Raman   | 2024-01-22 |
|          2 | Bikash       | bikash@email.com | Drama Club  | R303      | Mr. Kiran   | 2024-01-25 |
|          6 | Pooja        | pooja@email.com  | Sports Club | R202      | Ms. Sita    | 2024-01-27 |
|          3 | Nisha        | nisha@email.com  | Coding Club | Lab1      | Mr. Anil    | 2024-01-28 |
|          7 | Aman         | aman@email.com   | Coding Club | Lab1      | Mr. Anil    | 2024-01-30 |
+------------+--------------+------------------+-------------+-----------+-------------+------------+
10 rows in set (0.01 sec)
```

Figure 54: Repetition of data

After normalization, student information is stored in the student table and club information is stored in club table

```
mysql> select * from student_3nf
    -> ;
+------------+--------------+------------------+
| student_id | student_name | email            |
+------------+--------------+------------------+
|          1 | Asha         | asha@email.com   |
|          2 | Bikash       | bikash@email.com |
|          3 | Nisha        | nisha@email.com  |
|          4 | Rohan        | rohan@email.com  |
|          5 | Suman        | suman@email.com  |
|          6 | Pooja        | pooja@email.com  |
|          7 | Aman         | aman@email.com   |
|          8 | Peina        | peina@email.com  |
+------------+--------------+------------------+
8 rows in set (0.01 sec)

mysql> select * from club_3nf
    -> ;
+---------+---------------+-----------+-------------+
| club_id | club_name     | club_room | club_mentor |
+---------+---------------+-----------+-------------+
|       1 | Music Club    | R101      | Mr. Raman   |
|       2 | Sports Club   | R202      | Ms. Sita    |
|       3 | Drama Club    | R303      | Mr. Kiran   |
|       4 | Coding Club   | Lab1      | Mr. Anil    |
|       9 | Robotics Club | Lab 204   | Mr. Shyam   |
+---------+---------------+-----------+-------------+
5 rows in set (0.01 sec)
```

Figure 55: Normalized data

We can see in figure 55, normalization solves the data redundancy problem. The third membership table stores the relationship between student and club. Because of this data is not repeated multiple times.

```
mysql> select * from membership_3nf;
+---------------+------------+---------+------------+
| membership_id | student_id | club_id | join_date  |
+---------------+------------+---------+------------+
|             1 |          1 |       1 | 2024-01-10 |
|             2 |          1 |       2 | 2024-01-15 |
|             3 |          2 |       3 | 2024-01-25 |
|             4 |          2 |       2 | 2024-01-12 |
|             5 |          3 |       4 | 2024-01-28 |
|             6 |          3 |       1 | 2024-01-20 |
|             7 |          4 |       3 | 2024-01-18 |
|             8 |          5 |       1 | 2024-01-22 |
|             9 |          6 |       2 | 2024-01-27 |
|            10 |          7 |       4 | 2024-01-30 |
+---------------+------------+---------+------------+
10 rows in set (0.00 sec)
```

Figure 56: Relationship of Student_id and club_id

Data accuracy.

Normalization improves data accuracy as each piece of data is stores in one place. For example, in us unnormalized table, if we update student's email then it could cause errors if all rows are not updated correctly. Whereas in our normalized database, changes are made in single table, so data remains correct throughout the data base.

```
mysql> SELECT student_name, email, club_name
    -> FROM club_members_unf
    -> WHERE student_name = 'Asha';
+--------------+----------------+-------------+
| student_name | email          | club_name   |
+--------------+----------------+-------------+
| Asha         | asha@email.com | Music club  |
| Asha         | asha@email.com | Sports club |
+--------------+----------------+-------------+
2 rows in set (0.00 sec)
```

Figure 57: Asha old email

Initially Asha has this email. If we want to update the that email to new email, we must make sure all rows are updated carefully.

```
mysql> UPDATE club_members_unf
    -> SET email = 'asha_new@gmail.com'
    -> WHERE student_name = 'Asha'
    -> AND club_name = 'Music Club';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT student_name, email, club_name
    -> FROM club_members_unf
    -> WHERE student_name = 'Asha';
+--------------+--------------------+-------------+
| student_name | email              | club_name   |
+--------------+--------------------+-------------+
| Asha         | asha_new@gmail.com | Music club  |
| Asha         | asha@email.com     | Sports club |
+--------------+--------------------+-------------+
2 rows in set (0.00 sec)
```

Figure 58: Asha updated email

In figure 58, we can see that Asha one email is updated but email linked to sports club is not updated. If we have normalized the data, then this problem would not raise. Below is the updated email of Asha in normalized table and

```
mysql> UPDATE student_3nf
    -> SET email = 'asha_new@gmail.com'
    -> WHERE student_name = 'Asha';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT s.student_name, s.email, c.club_name
    -> FROM membership_3nf m
    -> JOIN student_3nf s ON m.student_id = s.student_id
    -> JOIN club_3nf c ON m.club_id = c.club_id
    -> WHERE s.student_name = 'Asha';
+--------------+--------------------+-------------+
| student_name | email              | club_name   |
+--------------+--------------------+-------------+
| Asha         | asha_new@gmail.com | Music Club  |
| Asha         | asha_new@gmail.com | Sports Club |
+--------------+--------------------+-------------+
2 rows in set (0.01 sec)
```

Figure 59: Asha updated email in 3NF

From figure 59, we can see that we have updated the email in student table and it automatically updated the Asha email in every club. Therefore, normalization prevents inconsistency and improves the data accuracy.

Necessity of Join Operation after normalization

When we normalized the table then the data are stored in the different tables to avoid repetition. When we separate the data, we cannot directly view data together. We need join operation to combine the data from multiple tables to get the meaningful information. (Calise, 2023).

For example, when we normalized the given unnormalized data, we have created three separated table students table, club tables and membership tables. In student table only student information is shown and in club table only club information.

```
mysql> select * from student_3nf
    -> ;
+------------+--------------+------------------+
| student_id | student_name | email            |
+------------+--------------+------------------+
|          1 | Asha         | asha@email.com   |
|          2 | Bikash       | bikash@email.com |
|          3 | Nisha        | nisha@email.com  |
|          4 | Rohan        | rohan@email.com  |
|          5 | Suman        | suman@email.com  |
|          6 | Pooja        | pooja@email.com  |
|          7 | Aman         | aman@email.com   |
|          8 | Peina        | peina@email.com  |
+------------+--------------+------------------+
8 rows in set (0.01 sec)

mysql> select * from club_3nf
    -> ;
+---------+---------------+-----------+-------------+
| club_id | club_name     | club_room | club_mentor |
+---------+---------------+-----------+-------------+
|       1 | Music Club    | R101      | Mr. Raman   |
|       2 | Sports Club   | R202      | Ms. Sita    |
|       3 | Drama Club    | R303      | Mr. Kiran   |
|       4 | Coding Club   | Lab1      | Mr. Anil    |
|       9 | Robotics Club | Lab 204   | Mr. Shyam   |
+---------+---------------+-----------+-------------+
5 rows in set (0.01 sec)
```

Figure 60: Student and Club table

As we can see in figure above due to normalization data is separated due to this, we cannot find out directly which student is the part of which club. So, in this situation Join operation is needed to retrieve the meaningful information.

```
mysql> SELECT
    ->     s.student_name,
    ->     c.club_name,
    ->     m.join_date
    -> FROM membership_3nf m
    -> JOIN student_3nf s ON m.student_id = s.student_id
    -> JOIN club_3nf c ON m.club_id = c.club_id;
+--------------+-------------+------------+
| student_name | club_name   | join_date  |
+--------------+-------------+------------+
| Asha         | Music Club  | 2024-01-10 |
| Asha         | Sports Club | 2024-01-15 |
| Bikash       | Drama Club  | 2024-01-25 |
| Bikash       | Sports Club | 2024-01-12 |
| Nisha        | Coding Club | 2024-01-28 |
| Nisha        | Music Club  | 2024-01-20 |
| Rohan        | Drama Club  | 2024-01-18 |
| Suman        | Music Club  | 2024-01-22 |
| Pooja        | Sports Club | 2024-01-27 |
| Aman         | Coding Club | 2024-01-30 |
+--------------+-------------+------------+
10 rows in set (0.02 sec)
```

Figure 51: Joined table of student and table

The above joined query is used to show the student and club relationship while membership is used to handle the relationship. Without join query we cannot get this useful information. Therefore, join operation helps to retrieve the meaningful information form the database.

Sharma, N. (2024) *What is data redundancy: Benefits, drawbacks, and management strategies*, *Analytics Vidhya*. Available at: https://www.analyticsvidhya.com/blog/2023/09/what-is-data-redundancy/#h-what-is-data-redundancy (Accessed: 04 February 2026).

Jha, N. (2024) *Anomalies in SQL. in SQL databases, anomalies refer to… | by Nirajan Jha | Medium*, *Anomalies in SQL*. Available at: https://medium.com/@nirajan_DataAnalyst/anomalies-in-sql-04650e07dd8d (Accessed: 04 February 2026).

Chris, K. (2022, December 21). *Database Normalization – Normal Forms 1NF 2NF 3NF Table examples*. freeCodeCamp.org. https://www.freecodecamp.org/news/database-normalization-1nf-2nf-3nf-table-examples/

Kitthu, H. A. (2025, September 17). *ER Diagrams in DBMS: Entity Relationship Diagram Model*. Simplilearn.com. https://www.simplilearn.com/tutorials/sql-tutorial/er-diagram-in-dbms#what_is_an_er_diagram

Kumar, A. (2024, May 15). SQL Commands Guide: DDL, DML, DCL, & TCL | K21Academy. *K21 Academy*. https://k21academy.com/azure-data/sql-commands/#:~:text=These%20SQL%20commands%20are%20primarily,Transaction%20Control%20Language%20(TCL)

Krantz, T., & Jonker, A. (2025, November 17). What is Data Redundancy ? *IBM*. https://www.ibm.com/think/topics/data-redundancy#:~:text=Authors,while%20recovery%20focuses%20on%20restoration.

Calise, M. (2023, March 1). *Data Normalization and Table Joins: a short guide to keep your database clean and your tables rich!* Nimbus Intelligence.

https://nimbusintelligence.com/2023/03/data-normalization-and-table-joins-a-short-guide-to-keep-your-database-clean-and-your-tables-rich